ASSIGNMENT 3 – TASK 1

LINKING, LOADING AND ADDRESS BINDING

GitHub Repository Link: _____

1. Introduction and Problem Context

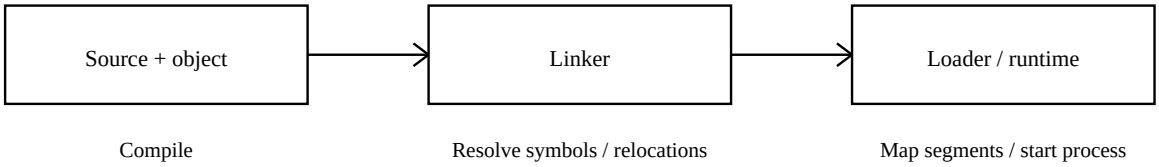
The task describes a bank whose branch software is built from multiple modules and libraries. When a security fix is made to an encryption library, every branch currently receives a new 200 MB executable. The objective is to compare static and dynamic linking, inspect loaded addresses, measure startup and memory behaviour, demonstrate independent shared-library replacement, and evaluate loading the library only when it is needed.

The supplied implementation uses a small demonstration library with two compatible versions. Its function performs a deterministic bitwise transform so that a library update creates an observable output change. It is deliberately not a cryptographic routine and must not be represented as real encryption.

2. Implementation Design

The same application source, bank_app.c, calls two exported library functions: bank_transform() and bankcrypto_version(). The source is compiled in two ways. The static build links libbankcrypto.a and the C runtime statically. The dynamic build links to libbankcrypto.so.1 and uses an executable-relative RUNPATH so the example can run from the Task01 folder.

Linking and loading flow



Static code is copied into the executable; dynamic code is mapped from a shared object.

A separate bank_app_dlopen.c demonstrates the alternative design: the application starts without an encryption-library dependency and loads the shared object only when the user requests the optional feature. It resolves function addresses using dlsym().

3. Static and Dynamic Linking

Aspect	Static linking	Dynamic linking
When library code is incorporated	Library object code is copied into the executable during linking.	The executable records a dependency on a shared object; the loader maps that object at program startup.
Executable file size	Usually larger because required library code is included.	Usually smaller because library code is stored in the shared object.
Runtime dependency	Does not require the application's custom .so file; the static executable contains the library code.	Requires a compatible shared library at runtime.
Updating a library	Application must be relinked and redistributed to include a new library version.	A compatible shared object can be updated independently of the executable.

Aspect	Static linking	Dynamic linking
Memory sharing	Executable pages are mapped per process; the system can share file-backed clean pages, but the library is not a separately named shared object dependency.	Read-only clean pages from the same shared object can be shared by processes mapping that file.
Failure mode	Library update is not picked up until the application is rebuilt/redeployed.	Missing or incompatible dependency can prevent the loader from starting the program.

The comparison must use the actual file sizes and ldd output produced on the target machine. Static and dynamic executable sizes depend on the compiler, C library, build flags, target architecture and which symbols are pulled into the final binary.

4. Linker and Loader Roles

Linker: combines object files and libraries, resolves external symbol references, performs relocation, and produces the executable. For a static build, it incorporates required library object code into the executable. For a dynamically linked build, it records the shared-library dependency and required symbol references.

Loader: creates the process image, maps executable segments and required shared objects into virtual memory, performs or arranges required relocations, creates the initial stack and process environment, and transfers control to the program entry point. The dynamic loader resolves runtime shared-library dependencies.

5. Address Binding

Binding stage	Explanation	Implication
Compile-time binding	If the final memory location is known in advance, the compiler/assembler can generate absolute addresses.	Moving the program to a different location requires rebuilding or relocating the code.
Load-time binding	The compiler produces relocatable code; the loader chooses a load address and applies relocation when loading.	The program can be placed at different locations on different executions, but its address mapping is fixed for that loaded image.
Execution-time binding	Address translation is performed while the program executes, typically using hardware address translation such as an MMU.	A process can be moved or mapped dynamically while executing, subject to the operating system and hardware support.

On modern systems, each process normally uses a virtual address space. The kernel and memory-management hardware map virtual pages to physical frames. The addresses printed by /proc/self/maps are process virtual address ranges, not physical RAM addresses.

6. Position-Independent Code (PIC)

Position-independent code is generated so that a shared library can execute correctly regardless of the virtual address at which it is mapped. The sample shared library is compiled with -fPIC and linked as a shared object. PIC commonly uses relative addressing and indirection tables such as the Global Offset Table and Procedure Linkage Table to access data and resolve external functions without requiring the library to be built for one fixed address.

PIC is important for shared libraries because multiple processes can map the same read-only code pages at different virtual addresses. Address Space Layout Randomisation (ASLR) can choose different base addresses for different processes.

7. Experimental Investigation

7.1 Executable sizes and shared-library dependencies

The run script records the byte sizes of bank_app_static, bank_app_dynamic and bank_app_dlopen in executable_sizes.txt. It records ldd output for the dynamic executable in ldd_dynamic.txt. A fully static executable is expected to be reported by ldd as “not a dynamic executable”.

Build	File size (bytes) – fill from executable_sizes.txt	Dependency observation
Static executable	_____	Expected: not a dynamic executable
Dynamic executable	_____	Lists libbankcrypto.so.1 and runtime dependencies
On-demand dlopen executable	_____	No link-time dependency on libbankcrypto.so.1

7.2 Loaded addresses using /proc/self/maps

bank_app.c opens /proc/self/maps and displays the mappings for the program, libbankcrypto.so.1, libc and the dynamic loader. The static version shows its program mappings but does not show libbankcrypto.so.1 as a separate mapped object. The dynamic version shows the shared library as a separate mapping. Virtual addresses can change between runs because of ASLR.

7.3 Startup-time measurement

The script launches each executable 30 times and records average milliseconds per launch in startup_time.txt. The measurement is a repeatable local process-launch comparison; it includes shell/command-launch overhead in both cases, so it should be interpreted as a comparative measurement rather than a precise isolated loader benchmark.

Version	Average startup time (ms/launch)	Method
Static	_____	30 launches; total elapsed time divided by 30
Dynamic	_____	30 launches; total elapsed time divided by 30

7.4 Memory measurement

The script records peak resident set size using /usr/bin/time -v and captures live VmSize, VmRSS, RssAnon, RssFile and VmData fields from /proc//status while the program is held open. Peak RSS and live RSS are different measures. Dynamic processes may show a larger raw RSS because shared libraries are included in the process’s RSS; proportional set size (PSS) or Shared_Clean is more informative when discussing page sharing.

Version	Peak RSS (KB)	Live VmRSS (KB)	Live VmSize (KB)
Static	_____	_____	_____
Dynamic	_____	_____	_____

7.5 Replace the shared library without rebuilding

The script runs bank_app_dynamic with library version 1, builds version 2 with the same SONAME and exported function interface, and replaces only lib/libbankcrypto.so.1. It then reruns the same dynamic executable. The output should change from bankcrypto-library-v1 to bankcrypto-library-v2 even though bank_app_dynamic is not recompiled. The statically linked executable should continue to print bankcrypto-library-v1 because its library implementation was incorporated at link time.

Test	Expected observation	Actual evidence file
Dynamic before replacement	bankcrypto-library-v1	dynamic_before_replacement.txt
Dynamic after replacement	bankcrypto-library-v2	dynamic_after_replacement.txt
Static after replacement	Still bankcrypto-library-v1	static_after_replacement.txt

The script restores the original v1 shared object after the replacement experiment so the remaining tests use the original library.

7.6 Run two copies and identify shared pages

Two dynamic application copies are started concurrently and held open. The script reads the libbankcrypto.so.1 entries in each process's /proc//smaps and totals Size, RSS, PSS, Shared_Clean and Private_Clean. Non-zero Shared_Clean values for read-only file-backed library mappings indicate pages that are currently mapped as clean shared pages. The two processes need not map the library at the same virtual address.

Process	Shared_Clean for libbankcrypto.so.1 (KB)	PSS for that mapping (KB)
Copy A	_____	_____
Copy B	_____	_____

7.7 Missing shared-library test

The script copies the dynamic executable into an isolated folder without the required library and runs it with an empty LD_LIBRARY_PATH. The dynamic loader should print an error similar to "libbankcrypto.so.1: cannot open shared object file" and the program will fail before entering main(). The exact wording depends on the platform.

8. Alternative Concept: Load the Library Only When Needed

Instead of linking libbankcrypto.so.1 as a startup dependency, the program can call dlopen() when the encryption feature is requested, resolve bank_transform and bankcrypto_version with dlsym(), use the functions, and call dlclose() when the handle is no longer needed. The supplied bank_app_dlopen.c implements this sequence.

Aspect	Dynamic link at startup	On-demand dlopen()
Startup	Required library is located and mapped before main().	Application can start without mapping the optional library.
Memory when feature unused	Shared object mappings and associated relocation state are present from startup.	Optional library need not be mapped until requested.
First feature use	Functions are already available after startup relocation.	First use incurs dlopen, relocation, symbol lookup and possible page-fault cost.

Aspect	Dynamic link at startup	On-demand dlopen()
Missing library	Loader prevents application startup if dependency is required.	Application can catch dlopen failure, report the issue and disable the optional feature.
Implementation complexity	Simpler call path after startup.	Requires handle management, dlsym error checks, function-pointer handling and dlclose.

The alternative is attractive if encryption is rarely used or is an optional feature: startup can be faster and the library's pages need not be mapped until requested. If the feature is required for every transaction, lazy loading merely moves cost to the first request and may create an undesirable latency spike.

8.1 Complexity and resource comparison

The transformation function itself performs a constant number of operations, so its per-call computational work is O(1). The extra work of loading on demand depends on library size, number of relocations and symbol lookups. A useful high-level model is O(L + R + S), where L represents library mapping/page-in work, R is the number of relocations and S is the symbol-resolution work; exact loader behaviour depends on the platform and symbol-table implementation. Subsequent calls use the resolved function pointer and do not repeat the full load operation.

In space terms, on-demand loading avoids mapping the optional library while it is unused (approximately O(0) additional library mapping for that feature). When loaded, its mapped code/data footprint is proportional to the pages required from the library, O(L). A startup-linked dynamic version maps its dependency during process initialization. Static linking includes the library code in the executable image; pages are brought into memory as required by normal demand paging.

9. Evaluation and Recommendation

For the banking scenario, dynamic linking is generally preferable for a library that is maintained and updated independently across many branches. A compatible security-library update can be deployed as a shared-library replacement without rebuilding every application executable, and read-only clean pages can be shared by concurrently running processes. The deployment process must still validate ABI compatibility, library versioning, package integrity and rollback behaviour.

Static linking provides a self-contained executable and avoids a missing custom shared-library dependency, but every branch must receive a rebuilt executable to incorporate the library update. It also tends to increase executable file size. A missing or incompatible dynamically linked library can stop the application at startup, so production systems need dependency packaging and health checks.

If the encryption feature is optional or infrequently used, on-demand dlopen() can reduce startup mapping and memory overhead when it is unused. If encryption is mandatory for all banking operations, startup dynamic linking is simpler and ensures the dependency is checked before the application begins serving transactions. A production banking system must use a vetted cryptographic library; the demonstration transform in this task is not secure.

10. Conclusion

The experiments distinguish static linking, startup dynamic linking and on-demand loading. They show how the linker resolves symbols, how the loader maps executables and shared objects, how /proc/self/maps exposes virtual address ranges, how a compatible shared library can be replaced without rebuilding a dynamically linked application, how clean file-backed pages can be shared across processes, and how

missing dependencies affect startup. For a bank that updates a common library across many branches, a dynamically linked design is a practical default, with on-demand loading suitable when the feature is genuinely optional. The final recommendation should be supported by the local size, startup-time and memory measurements recorded during execution.

11. Execution Procedure and Evidence

1. Save bank_app.c, bankcrypto_v1.c, bankcrypto_v2.c, bank_app_dlopen.c and run_task1.sh in one Task01 folder.
2. Open Ubuntu/WSL in that folder.
3. Run chmod +x run_task1.sh.
4. Run ./run_task1.sh.
5. Review executable_sizes.txt, ldd_dynamic.txt and the /proc/self/maps outputs.
6. Review startup_time.txt, memory_summary.txt and live_memory.txt.
7. Verify the v1-to-v2 replacement, shared_pages.txt and missing_library_output.txt.
8. Capture screenshots and preserve the generated text files in the Task01 folder.

Evidence checklist: static/dynamic executable sizes; ldd dependency list; /proc/self/maps addresses; startup and memory results; no-recompile shared-library replacement; two-process Shared_Clean/PSS evidence; and missing-library behaviour.

Source alignment: Q1 – Linking, Loading and Address Binding, 6 Marks, K5 – Evaluate, page 23 of the supplied Assignment 3 question paper.